Applied Mathematics and Computation 205 (2008) 98–108

Contents lists available at ScienceDirect

Applied Mathematics and Computation

journal homepage: www.elsevier.com/locate/amc

Particle swarm optimization with FUSS and RWS for high dimensional functions

Zhihua Cui [a,b,*], Xingjuan Cai [b], Jianchao Zeng [b], Guoji Sun [a]

[a] State Key Laboratory for Manufacturing Systems Engineering, Xi'an Jiaotong University, Xi'an 710049, PR China
[b] Division of System Simulation and Computer Application, Taiyuan University of Science and Technology, Taiyuan 030024, PR China

ARTICLE INFO

Keywords:
Particle swarm optimization
Fitness uniform selection strategy
Random walk strategy

ABSTRACT

High dimensional optimization problems play an important role in many complex engineering area. Though many variants of particle swarm optimization (PSO) have been proposed, however, most of them are tested and compared with dimension no larger than 300. Since numerical problem with high-dimension maintains a large linkage and correlation among different variables, and the number of local optimum increases significantly with different dimensions, this paper proposes a novel variant of PSO aiming to provide a balance between exploration and exploitation capability. Firstly, the fitness uniform selection strategy (FUSS) with a weak selection pressure is incorporated into the standard PSO. Secondly, "random walk strategy" (RWS) with four different form, is designed to further enhance the exploration capability to escaping from a local optimum. Finally, the proposed PSO combined with FUSS and RWS is applied to seven famous high dimensional benchmark with the dimension up to 3000. Simulation results demonstrate good performance of the new method in solving high dimensional multi-modal problems when compared with two other variants of the PSO.

1. Introduction

With the industrial and scientific development, many new optimization problems are needed to be solved. Several of them are complex multi-modal, high dimensional, non-differential problems. Therefore, some new optimization techniques have been designed, such as genetic algorithm [1], ant colony optimization [2], etc. However, due to the large linkage and correlation among different variables, these algorithms are easily trapped into a local optimum and failed to obtain the reasonable solution.

Particle swarm optimization (PSO) [3,4] is a population-based, self-adaptive search optimization method motivated by the observation of simplified animal social behaviors such as fish schooling, bird flocking, etc. It is becoming very popular due to its simplicity of implementation and ability to quickly converge to a reasonably good solution [5–7].

In a PSO system, multiple candidate solutions coexist and collaborate simultaneously. Each solution called a "particle", flies in the problem search space looking for the optimal position to land. A particle, as time passes through its quest, adjusts its position according to its own "experience" as well as the experience of neighboring particles. Tracking and memorizing the best position encountered build particle's experience. For that reason, PSO possesses a memory (i.e. every particle remembers the best position it reached during the past). PSO system combines local search method (through self experience) with global search methods (through neighboring experience), attempting to balance exploration and exploitation.

* Corresponding author. Address: State Key Laboratory for Manufacturing Systems Engineering, Xi'an Jiaotong University, Xi'an 710049, PR China.
E-mail address: cui_zhi_hua_7645@sohu.com (Z. Cui).

0096-3003/$ - see front matter © 2008 Elsevier Inc. All rights reserved.
doi:10.1016/j.amc.2008.05.147

A particle status on the search space is characterized by two factors: its position and velocity, which are updated by following equations:

$$\overrightarrow{v_j}(t+1) = w\overrightarrow{v_j}(t) + c_1 r_1(\overrightarrow{p_j}(t) - \overrightarrow{x_j}(t)) + c_2 r_2(\overrightarrow{p_g}(t) - \overrightarrow{x_j}(t)), \quad (1)$$

$$\overrightarrow{x_j}(t+1) = \overrightarrow{x_j}(t) + \overrightarrow{v_j}(t+1), \quad (2)$$

where $\overrightarrow{v_j}(t)$ and $\overrightarrow{x_j}(t)$ represent the velocity and position vectors of particle j at time t, respectively. $\overrightarrow{p_j}(t)$ means the best position vector which particle j had been found, as well as $\overrightarrow{p_g}(t)$ denotes the corresponding best position found by the whole swarm. Cognitive coefficient c_1 and social coefficient c_2 are constants known as acceleration coefficients, and r_1 and r_2 are two separately generated uniformly distributed random numbers in the range $[0, 1]$.

The first part of (1) represents the previous velocity, which provides the necessary momentum for particles to roam across the search space. The second part, known as the "cognitive" component, represents the personal thinking of each particle. The cognitive component encourages the particles to move toward their own best positions found so far. The third part is known as the "social" component, which represents the collaborative effect of the particles, in finding the global optimal solution. The social component always pulls the particles toward the global best particle found so far.

Since particle swarm optimization is a new swarm intelligent technique, many researchers focus their attentions to this new area. One famous improvement is the introduction of the inertia weight [8], similarly with temperature schedule in the simulated annealing algorithm. Empirical results showed the linearly decreased setting of inertia weight can give a better performance, such as from 1.4 to 0 [8], and 0.9 to 0.4 [9,10]. In 1999, Suganthan [11] proposed a time-varying acceleration coefficients automation strategy in which both c_1 and c_2 are linearly decreased during the course of run. Simulation results show the fixed acceleration coefficients at 2.0 generate better solutions. Following Suganthan's method, Venter [12] found that the small cognitive coefficient and large social coefficient could improve the performance significantly. Further, Ratnaweera et al. [13] investigated a time-varying acceleration coefficients. In this automation strategy, the cognitive coefficient is linearly decreased during the course of run, however, the social coefficient is linearly increased inversely.

Hybrid with Kalman filter, Monson designed a new Kalman filter particle swarm optimization algorithm [14]. Similarly, Sun proposed a new quantum particle swarm optimization [15] in 2004. From the convergence point, Cui designed a global convergence algorithm – stochastic particle swarm optimization [16]. There are still many other modified methods, such as fast PSO [17], predicted PSO [18], etc. The details of these algorithms can be found in corresponding references.

The PSO algorithm has been empirically shown to perform well on many optimization problems. However, it may easily get trapped in a local optimum when solving high dimensional multi-modal problems. With respect to the PSO model, several papers have been written on the subject to deal with premature convergence, such as the addition of a queen particle [19], the alternation of the neighborhood topology [20], the introduction of subpopulation and giving the particles a physical extension [21], etc. But their results are always tested on some famous benchmark with a middle dimension less than 300, and the optimization results are not very well for the high dimensional case. In order to improve PSO's performance on high dimensional multimodal problems, we present a new variant PSO combined with fitness uniform selection strategy (FUSS) and random walk strategy (RWS).

The rest of this paper is organized as follows: in Section 2, we summarize two significant previous developments to the standard PSO methodology. One method was used as the basis for our novel development, whereas the other one was selected as comparative measure of performance of the novel method proposed in this paper. In Section 3, we introduce the extension to PSO proposed in this paper. Experimental settings for the benchmarks and simulation strategies are explained in Section 4 and the results in comparison with the two previous developments are presented in Section 5.

2. Some previous work

In this paper, unconstrained optimization problems can be formulated as a D-dimensional minimization problem as follows:

$$\min f(\overrightarrow{x}), \overrightarrow{x} = [x_1, x_2, \ldots, x_D], \quad (3)$$

where D is the number of the parameters to be optimized.

In this section, we summarize two significant previous developments, which serve as both a basis for and performance gauge of the novel strategies introduced in this paper.

2.1. Hybrid particle swarm optimization

In PSO methodology, the personal best locations associated with each individual (e.g. $\overrightarrow{p_j}(t)$ for particle j) can be considered as additional population members that are manipulated according to the following definition:

$$\overrightarrow{p_j}(t+1) = \arg\min\{f(\overrightarrow{x_j}(t+1)), f(\overrightarrow{p_j}(t))\}. \quad (4)$$

This is similar to the collection of parents in an evolutionary computation with the exception that the linkage of parents in a particle swarm population is restricted by its position in the population while there is no such restriction for evolutionary computations. Meanwhile, there is a form of selection implicit in PSO although it is extremely weak [22].

In 1998, Angeline firstly incorporated a tournament selection method used in evolutionary programming into the PSO method (called hybrid PSO) [22]. This improvement is performed as follows. The fitness of each individual, based on its current position, is compared to the fitness of k other individuals and scores a point for each with worse fitness. The population is then sorted using this score with the individuals having the highest scores appearing at the head of the population. The fitness of the personal best positions for the individuals are not considered during the scoring or sorting of the individuals. Once the population is sorted, the current positions and velocities of the best half of the population are used to replace the positions and velocities of the worst half of the population leaving the personal best associated with each of the individuals unmodified.

With the addition of the selection method just described, the dynamics of hybrid PSO will be a composition of the dynamics of PSO and an evolutionary computation. In each generation, half of the individuals will be moved to positions of the search space that are relatively more optimal than their previous positions. The moved individuals will still contain their personal best points which will affect their next position.

While the difference between hybrid PSO and PSO is fairly minor, the addition of this selection method should provide hybrid PSO with a more exploitative search mechanism that should find better optima more consistently than PSO. Simulation results also demonstrated this point. This modification to the original PSO concept has been considered as the basis for the novel strategy introduced in this paper.

2.2. Diversity-guided particle swarm optimizer

In 2002, Riget and Vesterstrϕm [23] introduced the *attractive* and *repulsive* PSO (ARPSO) in trying to overcome the problem of premature convergence.

They defined the attraction phase merely as the standard PSO algorithm. The particles will then attract each other, since in general they attract each other in the standard PSO algorithm because of the information flow of good solutions between particles. However, the second phase repulsion, by "inverting" the velocity-update formula of the particles:

$$\overrightarrow{v_j}(t+1) = w\overrightarrow{v_j}(t) - \phi_1(\overrightarrow{p_j}(t) - \overrightarrow{x_j}(t)) - \phi_2(\overrightarrow{p_g}(t) - \overrightarrow{x_j}(t)), \tag{5}$$

where $\phi_1 = c_1 r_1$ and $\phi_2 = c_2 r_2$.

In the repulsion phase the individual particle is no longer attracted to, but instead repelled by the best known particle position $\overrightarrow{p_g}(t)$ and its own previous best position $\overrightarrow{p_j}(t)$.

In the attraction phase the swarm is contracting, and consequently the diversity is decreasing. When the diversity drops below a lower bound, d_{low}, we switch to the repulsion phase, in which the swarm expands due to the above inverted update-velocity formula (5). Finally, when a diversity of d_{high} is reached, we switch back to the attraction phase. The result of this is an algorithm that alternates between phases of exploiting and exploring – attraction and repulsion – low diversity and high diversity.

Now, a uniform velocity update equation combined with formula (1) and (5) is defined as follows:

$$\overrightarrow{v_j}(t+1) = w\overrightarrow{v_j}(t) + dir_j(t+1) \times [\phi_1(\overrightarrow{p_j}(t) - \overrightarrow{x_j}(t)) - \phi_2(\overrightarrow{p_g}(t) - \overrightarrow{x_j}(t))], \tag{6}$$

where the function $dir(t+1)$ is given by

$$dir_j(t+1) = \begin{cases} -1, & \text{if } dir_j(t) = 1 \quad \text{and} \quad diversity_j < d_{\text{low}}, \\ 1, & \text{if } dir_j(t) = -1 \quad \text{and} \quad diversity_j > d_{\text{high}}, \end{cases} \tag{7}$$

in which the function *diversity* is defined as follows:

$$diversity_j = \frac{1}{|S||L|} \times \sum_{k=1}^{|S|} \sqrt{\sum_{u=1}^{N} (p_{ku} - \overline{p}_u)^2}, \tag{8}$$

where S is the swarm, $|S|$ is the swarm size, $|L|$ is the length of longest the diagonal in the search space, N is the dimension of the problem, p_{ku} is the u'th value of the k'th particle and $\overline{p}_u$ is the u'th value of the average point $\overline{p}$.

Through empirical results with some of the well-known benchmark, it has been identified that ARPSO method shows a good performance for a high dimensional multi-modal functions. Therefore, this method was selected to compare the effectiveness of the novel PSO strategy introduced in this paper.

3. Proposed new developments

Although there are numerous variants for the PSO, premature convergence when solving high dimensional problems is still the main deficiency of the PSO. In the standard PSO, each particle learns from its *pbest* ($\overrightarrow{p_j}(t)$ for particle j) and *gbest* ($\overrightarrow{p_g}(t)$) simultaneously. Restricting the social learning aspect to only the *gbest* makes the original PSO converges fast. However, because all particles in the swarm learn from the *gbest* even if the current *gbest* is far from the global optimum, particles may easily be attracted to the *gbest* region and get trapped in a local optimum if the search environment is complex with

numerous local solutions. As $f(\vec{x}) = f([x_1, x_2, \ldots, x_D])$, the fitness value of a particle is possibly determined by values of all D parameters, and a particle that has discovered the region corresponding to the global optimum in some dimensions may have a low fitness value because of the poor solutions in the other dimensions. In order to make better use of the beneficial information, we propose an explicit selection strategy to improve the original PSO. In this new algorithm, fitness uniform selection strategy (FUSS) is applied to make a proper selection pressure to ensuring sufficient optimization progress on the one hand and preserving diversity measure to be able to escaping from local optima on the other hand. Then, several "random walk strategy" (RWS) are designed to further enhance the escaping capability. This method is discussed and demonstrated with significantly improved performances on solving high dimensional benchmarks in comparison to several other variants of the PSO.

3.1. Fitness uniform selection strategy

In evolutionary algorithms (EA), the fitness of a population increases with time by mutating and recombining individuals and by a biased selection of more fit individuals.

The standard selection schemes, proportionate, truncation, ranking and tournament selection all favor individuals of better fitness [24]. This is also true for less common schemes, like Boltzmann selection [25]. The fitness function is identified with the objective function (possibly after a monotone transformation). All these selection schemes have the property to increase the average fitness of a population, i.e. to evolve the population towards better fitness. If the selection pressure is too high, the EA gets stuck in a local optimum, since the genetic diversity rapidly decreases. The suboptimal genetic material which might help in finding the global optimum is deleted too rapidly (premature convergence). On the other hand, the selection pressure cannot be chosen arbitrarily low if we want EA to be effective.

Fitness uniform selection strategy [26] (FUSS) is based on the insight that it is not primarily interested in a population converging to optimum fitness, but only in a single individual of best fitness. The scheme automatically creates a suitable selection pressure. The FUSS is defined as follows: *if the lowest/highest fitness values in the current population P are* $f_{\min}/f_{\max}$, *we select a fitness value f uniformly in the interval* $[f_{\min}, f_{\max}]$. *Then, the individual* $i \in P$ *with fitness nearest to f is selected and a copy is added to P possibly after mutation and recombination.*

Although Angeline first proposed tournament selection in original PSO [22], the performance does not improve greatly with dimensionality from 10 to 30. Followed this idea, we incorporate the fitness uniform selection strategy into the PSO methodology to obtain a sufficient optimization process.

Let us suppose the individual pool at time t is $P(t) = [\vec{x_1}(t), \vec{x_2}(t), \ldots, \vec{x_M}(t)]$, where M denotes the number of individuals those may be chosen with nonzero probability (the individual pool is constructed with the random walk strategy introduced as follows). The upper bound $f_{\max}$ and lower bound $f_{\min}$ of the fitness interval are defined as follows:

$$f_{\max} = \max\{f(\vec{x_1}(t)), f(\vec{x_2}(t)), \ldots, f(\vec{x_M}(t))\}, \tag{9}$$

$$f_{\min} = \min\{f(\vec{x_1}(t)), f(\vec{x_2}(t)), \ldots, f(\vec{x_M}(t))\}. \tag{10}$$

Then, a random number f_{rand} is uniformly generated within interval $(f_{\min}, f_{\max})$, and the individual j is chosen by

$$\vec{x_j}(t) = \arg\min\{||f(\vec{x_k}(t)) - f_{\text{rand}}||_2, \quad k = 1, 2, \ldots, M\}, \tag{11}$$

where $\|\cdot\|_2$ represents the 2-norm.

This is the FUSS strategy to select one particle, and this process will continue until the predefined number of particles are chosen.

3.2. Random walk strategy

An individual pool is needed for FUSS selection strategy, to make a better selection effect, the number of candidates should be large enough, it means the number M should be no less than the swarm of particles. Therefore, some additional *shadow particles* are added to provide a precise distribution density estimation. These *shadow particles* simulate the animal random walk phenomenon.

In nature, the animal's seeking behavior can be roughly categorized into random walk and direct walk. If an animal does not find the food within a predefined time, the random walk strategy occurs to provide some probability to explore other region outside the region searched previously. If it detects some quarry information, the search phase will translate from "random walk" to "direct walk" to enlarge the successful predation probability.

Inspired by this phenomenon, we propose the random walk strategy (RWS). With RWS, each particle has an additional *shadow particle*. For convenience, the symbol $\vec{sx_j}(t)$ denotes the position information of j'th shadow particle, as well as $\vec{sv_j}(t)$ represents the velocity vector of j'th shadow particle.

To make a deeper insight of RWS, four different variants of random walk strategy are designed to illustrate the exploration capability to avoid premature convergence. The different velocity of *shadow particles* are defined as follows:

Strategy one:

$$\vec{sv_j}(t+1) = w\vec{v_j}(t) - c_1 r_1(\vec{p_j}(t) - \vec{x_j}(t)) - c_2 r_2(\vec{p_g}(t) - \vec{x_j}(t)). \tag{12}$$

Strategy two:

$$\overrightarrow{sv_j}(t+1) = \begin{cases} v_{\max} \times rand(), & \text{if } Rand() < 0.5, \\ -v_{\max} \times rand(), & \text{otherwise}, \end{cases} \tag{13}$$

where $rand()$ and $Rand()$ are two separately random numbers uniformly generated within $(0,1)$. $v_{\max}$ is a parameter to control excessive roaming of particles outside the search space.

Strategy three:

$$\overrightarrow{sv_j}(t+1) = \overrightarrow{v_j}(t+1) + rand(), \tag{14}$$

where $rand()$ is a random number uniformly generated within $(0,1)$.

Strategy four:

$$\overrightarrow{sv_j}(t+1) = \overrightarrow{v_j}(t+1) + Cauchy\_rand(), \tag{15}$$

where $Cauchy\_rand()$ is a random number satisfied with Cauchy distribution [27].

Then, the shadow particle's position vector $\overrightarrow{sx_j}(t+1)$ is given by

$$\overrightarrow{sx_j}(t+1) = \overrightarrow{x_j}(t) + \overrightarrow{sv_j}(t+1). \tag{16}$$

All these random walk strategies aim to explore other region to increase the probability escaping from a local optimum.

To further make the algorithm more effective, a probability threshold p_E is used to control the number of shadow particles. It provide a manner to accelerate the convergence speed during the last period. In this paper, we select the threshold p_E decreases linearly from 0.9 to 0.1. This setting only comes from the experimental test.

3.3. Pseudo-code for the PSO with FUSS and RWS

The pseudo-code for the PSO with FUSS is illustrated as follows:

Step1. Initializing the position vector and velocity vector of each component of each particle, determining the personal best location and the best location among the entire swarm;
Step2. Updating the velocity vector and position vector of each particle of the swarm according to Eqs. (1) and (2);
Step3. Updating the velocity and position information of each shadow particle with the threshold p_E according to formula (12)–(16) with respect to different random walk strategy;
Step4. Combining the total particles and corresponding shadow particles to consist an individual pool;
Step5. Applying FUSS to select N particles' current position and velocity information, where N is the swarm size;
Step6. Determining the personal best location and the best position of the entire swarm;
Step7. If the stop criteria is satisfied, output the obtained best solution; otherwise, go to Step 2.

4. Experimental settings for benchmark testing

4.1. Benchmarks

Seven of the well-known benchmark are used to evaluate the performance of our new developments introduced in this paper. These benchmarks are widely used in evaluating performance of PSO methods. They are:

Sphere Model:

$$f_1(x) = \sum_{j=1}^{n} x_j^2,$$

where $|x_j| \leqslant 100.0$, and

$$f_1(x^*) = f_1(0,0,\ldots,0) = 0.0.$$

Rosenbrock Function:

$$f_2(x) = \sum_{j=1}^{n-1} \left[100(x_{j+1} - x_j^2)^2 + (x_j - 1)^2\right],$$

where $|x_j| \leqslant 30.0$, and

$$f_2(x^*) = f_2(1,1,\ldots,1) = 0.0,$$

Quartic Function i.e. Noise:

$$f_3(x) = \sum_{j=1}^{n} j x_j^4 + rand(0,1),$$

where $|x_j| \leqslant 1.28$, $rand(0,1)$ is a uniformly distributed random number and

$$f_3(x^*) = f_3(0,0,\ldots,0) = 0.0.$$

Ackley Function:

$$f_4(x) = -20\exp\left(-0.2\sqrt{\frac{1}{n}\sum_{j=1}^{n} x_j^2}\right) - \exp\left(\frac{1}{n}\sum_{k=1}^{n}\cos 2\pi x_k\right) + 20 + e,$$

where $|x_j| \leqslant 32.0$, and

$$f_4(x^*) = f_4(0,0,\ldots,0) = 0.0.$$

Griewank Function:

$$f_5(x) = \frac{1}{4000}\sum_{j=1}^{30} x_j^2 - \prod_{j=1}^{30}\cos\left(\frac{x_j}{\sqrt{j}}\right) + 1,$$

where $|x_j| \leqslant 600.0$, and

$$f_5(x^*) = f_5(0,0,\ldots,0) = 0.$$

Penalized Function1:

$$f_6(x) = \frac{\pi}{30}\left\{10\sin^2(\pi y_1) + \sum_{i=1}^{n-1}(y_i-1)^2[1+10\sin^2(\pi y_{i+1})] + (y_n-1)^2\right\} + \sum_{i=1}^{n} u(x_i,10,100,4),$$

where $|x_j| \leqslant 50.0$, and

$$u(x_i,a,k,m) = \begin{cases} k(x_i-a)^m, & \text{if } x_i > a,\\ 0, & \text{if } -a \leqslant x_i \leqslant a,\\ k(-x_i-a)^m, & \text{if } x_i < -a, \end{cases}$$

$$y_i = 1 + \frac{1}{4}(x_i+1),$$

$$f_6(x^*) = f_6(1,1,\ldots,1) = 0.0.$$

Penalized Function2:

$$f_7(x) = 0.1\{\sin^2(3\pi x_1) + \sum_{i=1}^{n-1}(x_i-1)^2[1+\sin^2(3\pi x_{i+1})] + (x_n-1)^2[1+\sin^2(2\pi x_n)]\} + \sum_{i=1}^{n} u(x_i,5,100,4),$$

where $|x_j| \leqslant 50.0$, and

$$f_7(x^*) = f_7(1,1,\ldots,1) = 0.0.$$

Sphere, Rosenbrock and Quartic Function i.e. Noise are simple unimodal functions whereas others are multi-modal functions designed with a considerable amount of local minima. Simulation were carried out to find the global minimum of each function.

4.2. Simulation strategies

Simulations were carried out to observe the quality of the optimum solution of the new methods introduced, and compared with both the standard PSO (SPSO) and *attractive* and *repulsive* PSO (ARPSO). For convenience, the new methods with different random walk strategies are illustrated as MPSO1 (with formula (12) and (16)), MPSO2 (with formula (13) and (16)), MPSO3 (with formula (14) and (16)) and MPSO4 (with formula (15) and (16)), respectively.

All benchmarks were tested with dimensions 50, 100, 500, 1000, 1500, 2000, 2500, 3000. For each function, 50 trials were carried out and the average optimal value and the standard deviation (STD) are presented. The largest generation was set to 50 times the dimensionality of the test function, i.e. for dimension 3000, the number is $50 \times 3000 = 150,000$. The swarm size is 20.

The detailed parameter setting as follows: inertia weight is linearly decreased from 0.9 to 0.4, two accelerator coefficients are set to 2.0. Other parameter setting is set as same as the corresponding reference (for example, ARPSO is [23]).

5. Results from benchmark simulations

To make a precise comparison, the simulation is divided to two parts: firstly, we compare the proposed methods MPSO1 to MPSO4 for the total seven benchmarks. After the modified version with the best average performance is chosen, we compare it with SPSO and ARPSO. Finally, the conclusion is given.

5.1. Comparison of the proposed methods

In this section, we compare the proposed four random walk strategies, and the simulation results are listed as follows (see Figs. 1–7):

Figs. 1–7 provide a details of the comparison results among the four proposed methods. Since Sphere, Rosenbrock and Quartic Function i.e. Noise are unimodal benchmarks, there is an interesting evidence that the MPSO1 maintains nearly a linear speed with dimension increased except for Rosenbrock. For Rosenbrock, it's performance can also been viewed as a line for a large dimension no less than 500, although the performance increases significantly for a little dimension less than 500. Further, MPSO1 shows a significant better than other three variants no matter the average mean value or the standard deviation. Therefore, MPSO1 is the best choice for unimodal functions with four proposed RWS.

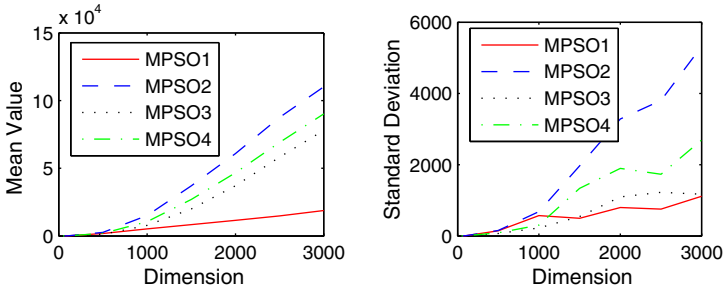

Fig. 1. Average mean value (left) and standard deviation (right) comparison results of sphere model.

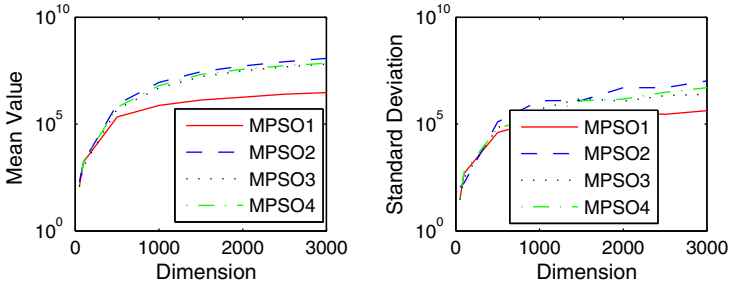

Fig. 2. Average mean value (left) and standard deviation (right) comparison results of Rosenbrock Function.

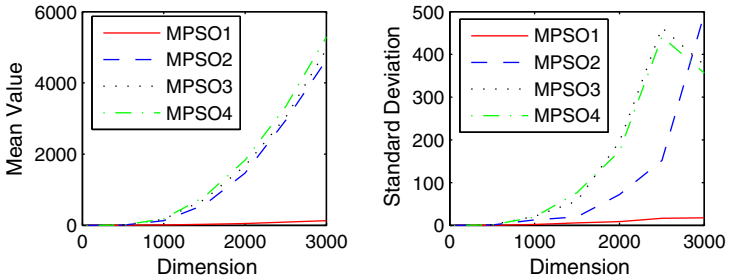

Fig. 3. Average mean value (left) and standard deviation (right) comparison results of Quartic Function i.e. Noise.

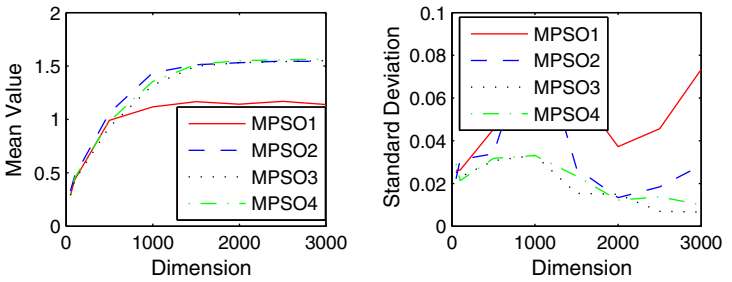

Fig. 4. Average mean value (left) and standard deviation (right) comparison results of Ackley Function.

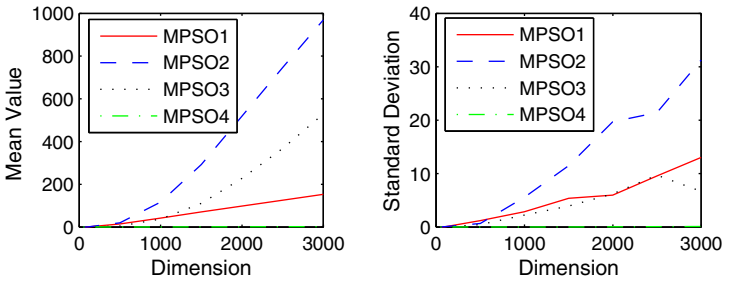

Fig. 5. Average mean value (left) and standard deviation (right) comparison results of Griewank Function.

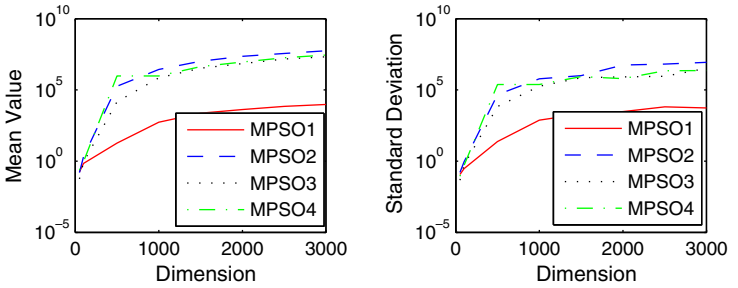

Fig. 6. Average mean value (left) and standard deviation (right) comparison results of Penalized Function1.

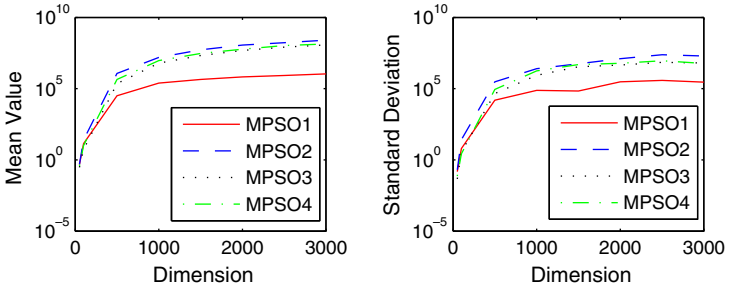

Fig. 7. Average mean value (left) and standard deviation (right) comparison results of penalized function2.

For Ackley and two Penalized functions, MPSO1 is also better than other variants. However, for Griewank function, MPSO1 is less than MPSO4 no matter the mean value and the standard deviation. For a large dimension, the term $\prod_{j=1}^{D} \cos\left(\frac{x_j}{\sqrt{j}}\right) \to 0$. In other words, Griewank is nearly approaching to a unimodal function for a large dimension. Therefore, MPSO1 is superior than other variants for the multi-modal functions with many local optima.

Now, we can see that MPSO1 is superior than other variants for both unimodal and multi-modal functions. This shows the directed escaping movement of strategy one (Eq. (12)) is better than the random movements (Strategy two, three and four). Further, strategy one can provide a large movement in some cases, compared with strategy three only with a small disturbance, this may provide a sufficient probability to escaping from a local optimum.

5.2. Comparison with SPSO and ARPSO

To make a deeper insight for the performance of MPSO1, we compare it with SPSO and ARPSO. Experimental results are listed as Figs. 8–14. The results show MPSO1 is superior than SPSO and ARPSO for average mean value with all seven benchmarks. However, for standard deviation, MPSO1 is not always better especially for Ackley. This phenomenon shows the FUSS is exactly an effective mechanism because the difference between MPSO1 and ARPSO is only the introduction of FUSS. In the first period, MPSO1 does not possess a more exploration capability, however, it can maintain this character in the latter search process. This may partly explain why MPSO1 achieves the best performance for a large dimensionality. In one word, MPSO1 maintains a power exploration capability, and achieves a better performance than other two algorithms.

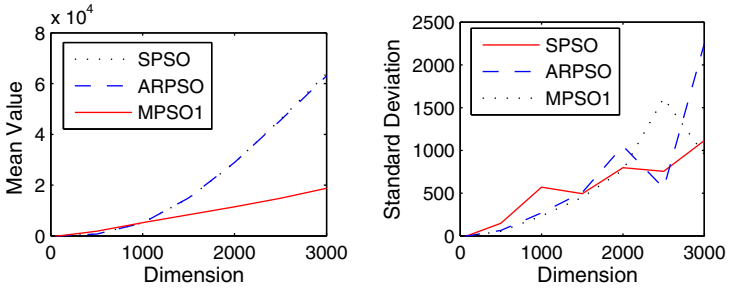

Fig. 8. Average mean value (left) and standard deviation (right) comparison results of Sphere Function.

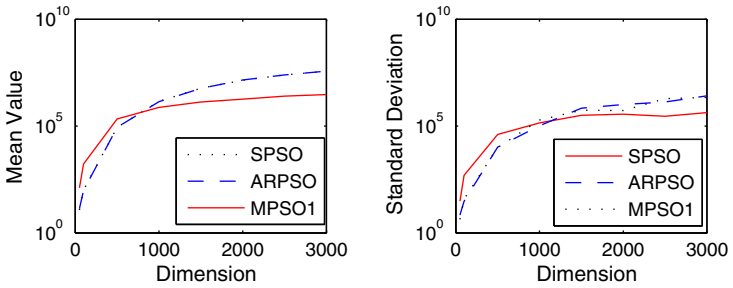

Fig. 9. Average mean value (left) and standard deviation (right) comparison results of Rosenbrock Function.

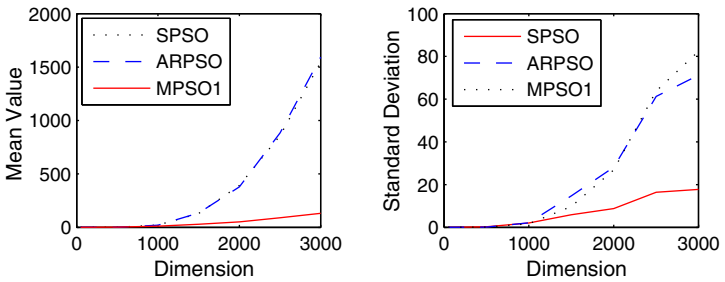

Fig. 10. Average mean value (left) and standard deviation (right) comparison results of Quartic Function i.e. Noise.

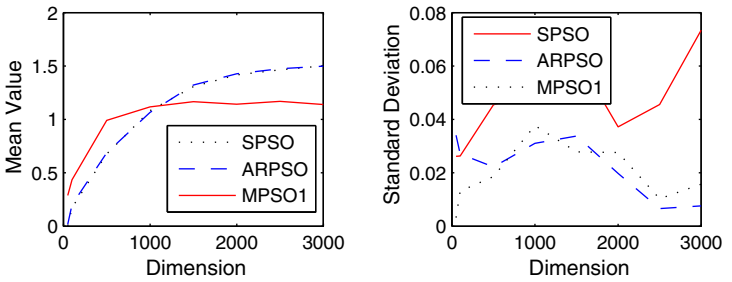

Fig. 11. Average mean value (left) and standard deviation (right) comparison results of Ackley Function.

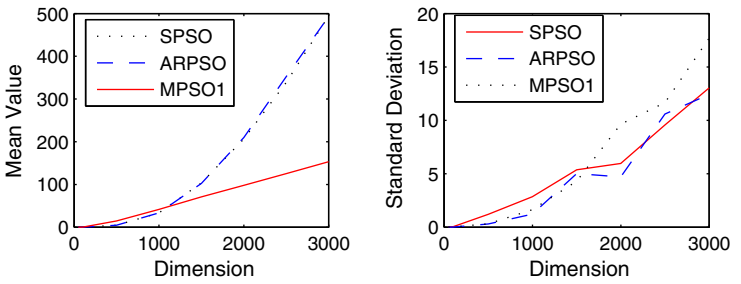

Fig. 12. Average mean value (left) and standard deviation (right) comparison results of Griewank Function.

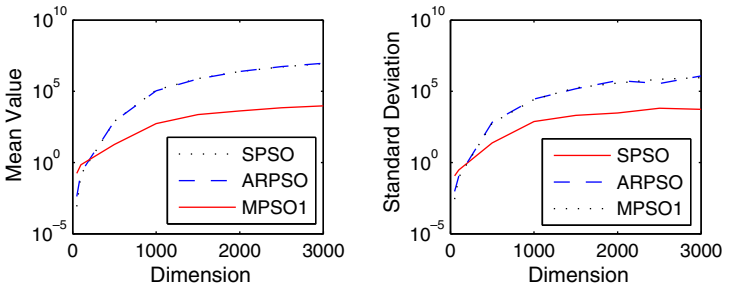

Fig. 13. Average mean value (left) and standard deviation (right) comparison results of Penalized Function1.

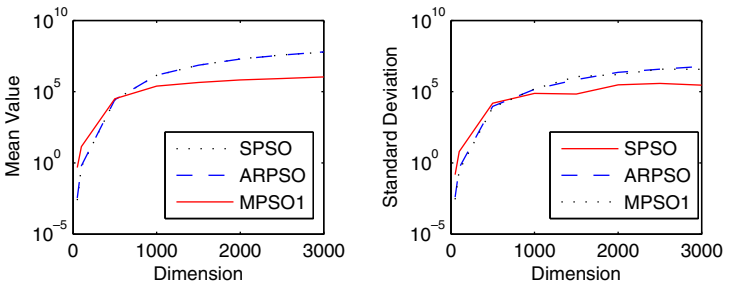

Fig. 14. Average mean value (left) and standard deviation (right) comparison results of Penalized Function2.

6. Conclusion

This paper proposes a new model incorporated with the fitness uniform selection strategy (FUSS) and random walk strategy (RWS). To make the new method effective, four different random walk strategies are designed and compared. Simulation

results show the first proposed technique is a robust optimization search method. There are still some further research topic need to be considered:

(1) Since the introduction of FUSS can result a complete different performance between MPSO1 and ARPSO, how to design a more powerful selection mechanism is an interesting research area.
(2) Because MPSO1 is superior than other three variants for all the benchmarks with the different disturbance size. How to make a proper dynamic selection of the movement size?
(3) Since we only consider the un-constraint numerical optimization problems, there is an interesting problem is to apply this hybrid method to solve constraint problem. Since in constraint problem, one of the important aspect is the swarm diversity, therefore, the hybrid method with FUSS may be effective. For example, we can apply FUSS to select the corresponding secondary (i.e. external) repository of particles [28].

Acknowledgement

This paper is supported by National Natural Science Foundation of China under Grant No. 60674104.

References

[1] J.H. Holland, Adaptation in Natural and Artificial Systems: An Introductory Analysis with Application to Biology, Control, and Artificial Intelligence, second ed., MIT Press, Cambridge, MA, 1992.
[2] M. Dorigo, L.M. Gambardella, Ant colony system: a cooperative learning approach to the traveling salesman problem, IEEE Transactions on Evolutionary Computation 1 (1) (1997) 53–66.
[3] R.C. Eberhart, J. Kennedy, A new optimizer using particle swarm theory, in: Proceedings of 6th International Symposium on Micro Machine and Human Science, 1995, pp. 39–43.
[4] J. Kennedy, R.C. Eberhart, Particle swarm optimization, Proceedings of IEEE International Conference on Neural Networks (1995) 1942–1948.
[5] H.Y. Shen, X.Q. Peng, J.N. Wang, Z.K. Hu, A mountain clustering based on improved PSO algorithm, Lecture Notes on Computer Science 3612, Changsha, China, 2005, pp. 477–481.
[6] R.C. Eberhart, Y. Shi, Extracting rules from fuzzy neural network by particle swarm optimization, in: Proceedings of IEEE International Conference on Evolutionary Computation, Anchorage, Alaska, USA, 1998.
[7] Q.Y. Li, Z.P. Shi, J. Shi, Z.Z. Shi, Swarm intelligence clustering algorithm based on attractor, Lecture Notes on Computer Science 3612, Changsha, China, 2005, pp. 496–504.
[8] Y. Shi, R.C. Eberhart, A modified particle swarm optimizer, in: Proceedings of the IEEE International Conference on Evolutionary Computation, Anchorage, Alaska, USA, 1998, pp. 69–73.
[9] Y. Shi, R.C. Eberhart, Parameter selection in particle swarm optimization, in: Proceedings of the 7th Annual Conference on Evolutionary Programming, 1998, pp. 591–600.
[10] Y. Shi, R.C. Eberhart, Empirical study of particle swarm optimization, in: Proceedings of the Congress on Evolutionary Computation (1999) 1945–1950.
[11] P.N. Suganthan, Particle swarm optimizer with neighbourhood operator, in: Proceedings of the Congress on Evolutionary Computation (1999) 1958–1962.
[12] G. Venter, Particle swarm optimization, in: Proceedings of 43rd AIAA/ASME/ASCE/AHS/ASC Structure, Structures Dynamics and Materials Conference, 2002, pp. 22–25.
[13] A. Ratnaweera, S.K. Halgamuge, H.C. Watson, Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients, IEEE Transactions on Evolutionary Computation 8 (3) (2004) 240–255.
[14] C.K. Monson, K.D. Seppi, The Kalman swarm: a new approach to particle motion in swarm optimization, in: Proceedings of the Genetic and Evolutionary Computation Conference, 2004, pp. 140–150.
[15] J. Sun, Particle swarm optimization with particles having quantum behavior, Proceedings of the IEEE Congress on Evolutionary Computation (2004) 325–331.
[16] Z.H. Cui, J.C. Zeng, A guaranteed global convergence particle swarm optimizer, in: Lecture Notes in Artificial Intelligence, vol. 3066, Sweden, 2004, pp. 762–767.
[17] Z.H. Cui, J.C. Zeng, G.J. Sun, A fast particle swarm optimization, International Journal of Innovative Computing, Information and Control 2 (6) (2006) 1365–1380.
[18] Z.H. Cui, X.J. Cai, J.C. Zeng, G.J. Sun, Predicted-velocity particle swarm optimization using game-theoretic approach, in: Lecture Notes in Computer Science, vol. 4115, Kunming, China, 2006, pp. 145–154.
[19] R. Mendes, J. Kennedy, J. Neves, The fully informed particle swarm: simper, maybe better, IEEE Transactions on Evolutionary Computation 8 (3) (2004) 204–210.
[20] J. Kennedy, Small worlds and mega-minds: effects of neighborhood topology on particle swarm performance, in: Proceedings of the 1999 Congress on Evolutionary Computation 15 (1999) 1931–1938.
[21] M. Løvbjerg, T.K. Rasmussen, T. Krink, Hybrid particle swarm optimiser with breeding and subpopulations, in: Proceedings of the Third Genetic and Evolutionary Computation Conference, 2001.
[22] P.J. Angeline, Using selection to improve particle swarm optimization, in: Proceedings of the IEEE International Conference on Evolutionary Computation, Anchorage, Alaska, USA, 1998, pp. 84–89.
[23] J. Riget, J.S. Vesterstrøm, A diversity-guided particle swarm optimizer – the ARPSO, EVALife Technical Report No. 2002-02, 2002.
[24] D.E. Goldberg, Genetic Algorithms in Search, Optimization and Machine Learning, Addison-Wesley, Reading, Mass, 1989.
[25] M. de la Maza, B. Tidor, An analysis of selection procedures with particular attention paid to proportional and Boltzmann selection, in: Proceedings of the 5th International Conference on Genetic Algorithms, San Mateo, CA, USA, 1993, pp. 124–131.
[26] M. Hutter, Fitness uniform selection to preserve genetic diversity, in: Proceedings of the IEEE International Conference on Evolutionary Computation (2002) 783–788.
[27] X. Yao, Y. Liu, G.M. Lin, Evolutionary programming made faster, IEEE Transactions on Evolutionary Computation 3 (1999) 82–102.
[28] A.C.C. Coello, G.T. Pulido, M.S. Lechuga, Handling multiple objectives with particle swarm optimization, IEEE Transactions on Evolutionary Computation 8 (3) (2004) 256–279.